

ADZE: Allelic Diversity Analyzer

Version 2.0

Zachary A. Szpiech

Mattias Jakobsson

Noah A. Rosenberg

Abstract

ADZE implements a rarefaction approach to estimating allelic richness, private allelic richness, and private allelic richness for combinations of populations, correcting for differences in sample size between populations. Version 2.0 computes the same estimators as version 1.0 — results are identical to the last bit — but reads its data in one pass, caches the rarefaction probabilities, parallelizes over loci, and replaces the parameter file with a conventional command-line interface. Section 9 lists everything that changed.

Contents

1	Introduction	3
1.1	Allelic richness	3
1.2	Private allelic richness	3
1.3	Private allelic richness for a combination of populations	4
1.4	Summary statistics	4
1.5	One gene copy	4
2	Getting started	5
2.1	Obtaining ADZE	5
2.2	Building	5
2.3	Running	5
3	Input	6
3.1	Choosing a format	6
3.2	STRUCTURE format	6
3.3	Dimensions are measured, not declared	7
3.4	VCF format	7
3.5	Groupings	8
3.6	Missing data and locus filtering	8
4	Parameters and options	9
4.1	Parameter file syntax	9
4.2	Reporting a single g	9
4.3	Option reference	9
4.3.1	Input options	10
4.3.2	Analysis options	11
4.3.3	Output options	12
4.3.4	Other options	12

5	Output	13
5.1	Allelic richness and private allelic richness	13
5.2	Private allelic richness of tuples	13
5.3	Locus-specific output	13
5.4	Filtered loci	13
5.5	Run summary	13
5.6	Tab-separated output	14
5.7	Exit status	14
6	Sliding windows	14
6.1	What a windowed statistic is here	14
6.2	Coordinates	14
6.3	Defining the windows	15
6.4	g is fixed across the scan	15
6.5	Output	15
7	Examples	16
7.1	The distributed example data set	16
7.2	A larger data set	16
7.3	Naming the tuples of interest	17
7.4	A genome scan	20
8	Performance and threading	21
9	Changes from version 1.0	21
9.1	Compatibility	21
9.2	Interface	22
9.3	Behaviour	22
9.4	Verification	23
	References	23

1 Introduction

The analysis of the distributions of alleles across populations is important for elucidating genetic diversity and population relationships. Two fundamental quantities for a population at a given locus are the number of distinct alleles in the population and the number of alleles found in that population and no other. Both are strongly dependent on sample size: a larger sample of gene copies is more likely to contain any given allele, so counts taken from samples of different sizes cannot be compared directly.

Rarefaction addresses this by computing, for a standardized sample size g , the expected number of alleles that *would* be observed if g gene copies were drawn from each population. The estimators ADZE computes are exact expectations under sampling without replacement, not resampling averages, so no random number generator and no seed are involved.

ADZE implements these computations for allelic richness, private allelic richness, and a generalization of private allelic richness to combinations of populations, as described in Szpiech et al. (2008).

1.1 Allelic richness

Consider a locus with I distinct alleles, and define N_{ij} as the number of copies of allele type i in a sample from population j . Let $N_j = \sum_{i=1}^I N_{ij}$ be the sample size of population j at the locus. The probability of finding no copies of allele type i in a subsample of g gene copies from population j is

$$Q_{ijg} = \frac{\binom{N_j - N_{ij}}{g}}{\binom{N_j}{g}}. \quad (1)$$

The probability of finding at least one copy of allele type i in a sample of size g from population j is $P_{ijg} = 1 - Q_{ijg}$, and

$$\hat{\alpha}_g^{(j)} = \sum_{i=1}^I P_{ijg} \quad (2)$$

is the estimated allelic richness of a sample of size g from population j (Hurlbert, 1971; Petit et al., 1998; Kalinowski, 2004). Equation 2 estimates the expected number of distinct alleles that will be observed in population j in a sample of size g .

1.2 Private allelic richness

Using this notation, the estimated private allelic richness for a sample of size g from population j is

$$\hat{\pi}_g^{(j)} = \sum_{i=1}^I \left[P_{ijg} \left(\prod_{\substack{j'=1 \\ j' \neq j}}^J Q_{ij'g} \right) \right], \quad (3)$$

where J is the total number of populations (Kalinowski, 2004). This formula sums over distinct allele types i the probability that a random subsample of size g from population j contains allele type i and that subsamples of size g from the remaining populations do not.

1.3 Private allelic richness for a combination of populations

Generalizing the concept of private allelic richness, we can consider the number of distinct alleles private to some combination of k populations selected from $\{1, 2, \dots, J\}$. Let $S = \{1, 2, \dots, J\}$ and let C_k be the set of all combinations of k elements from S ; there are $\binom{J}{k}$ of them. Labelling these combinations C_{km} , where m ranges from 1 to $\binom{J}{k}$, the estimated number of distinct alleles private to the m th combination of k populations, when samples of size g are drawn from each of the J populations, is

$$\hat{\pi}_{gk}^{(m)} = \sum_{i=1}^I \left[\left(\prod_{j \in C_{km}} P_{ijg} \right) \left(\prod_{j' \in S \setminus C_{km}} Q_{ij'g} \right) \right]. \quad (4)$$

Here $S \setminus C_{km}$ denotes the set S excluding the elements of C_{km} . For $k = 1$, $\hat{\pi}_{gk}^{(m)}$ reduces to private allelic richness as in Equation 3. For $k = J - 1$, Equation 4 can be considered a measure of “missing allelic richness”, and it reduces to

$$\hat{\mu}_g^{(j)} = \sum_{i=1}^I \left[Q_{ijg} \left(\prod_{\substack{j'=1 \\ j' \neq j}}^J P_{ij'g} \right) \right], \quad (5)$$

the expected number of alleles absent from population j but present in every other population.

1.4 Summary statistics

ADZE evaluates Equations 2–4 at each of many loci in a data set, and its primary method of reporting results gives the mean, variance, and standard error of the mean across these loci. Locus-specific estimates can also be written out (section 5.3). The mean across loci is calculated at each g and for each population or population grouping as

$$\bar{x} = \frac{1}{L} \sum_{\ell=1}^L x_{\ell}, \quad (6)$$

where L is the number of loci used in the calculation and x_{ℓ} is one of the statistics above evaluated at locus ℓ . The variance across loci is

$$s^2 = \frac{1}{L-1} \sum_{\ell=1}^L (x_{\ell} - \bar{x})^2, \quad (7)$$

and the standard error of the mean is

$$\text{SE} = \frac{s}{\sqrt{L}}. \quad (8)$$

A statistic is reported at a given g only if it is defined at that g for every locus used, which requires $g \leq N_j$ at each locus for the grouping concerned. With a single locus ($L = 1$) the variance and standard error are undefined and are reported as **nan**.

1.5 One gene copy

The sweep starts at $g = 1$, which is worth reading once because each statistic means something different there and one of them is a constant.

Allelic richness at $g = 1$ is exactly 1 at every locus: with $Q_{ji1} = (N_j - N_{ji})/N_j$ each allele contributes its own frequency p_{ji} , and the frequencies sum to 1. Drawing a single gene copy always yields one distinct allele, so the row is the estimator agreeing with a certainty rather than reporting anything about the data. It is still useful as a check: a locus that does not give 1 means the allele binning or the recurrence is wrong. Its variance across loci comes out as a number around 10^{-33} , the square of the rounding in a sum of ones, which is numerically zero.

Private allelic richness at $g = 1$ is $\sum_i p_{ji} \prod_{j' \neq j} (1 - p_{j'i})$: the chance that a single gene copy drawn from j carries an allele that single draws from every other grouping all miss. Note that every allele present in j contributes, weighted by how likely it is to be missed elsewhere — this is not the summed frequency of the alleles strictly private to j . The tuple statistic at $g = 1$ is the same form with the product of p taken over the tuple's members.

2 Getting started

2.1 Obtaining adze

Source is at <https://github.com/szpiech/ADZE>. Version 2.0 requires nothing but a C++17 compiler. Two libraries are used when the toolchain provides them, and the program builds and runs without either: OpenMP for `--threads`, and zlib for compressed input. Without zlib a `.gz` file is refused with a message rather than misread.

2.2 Building

With CMake:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
cmake --install build --prefix ~/.local      # optional
```

Or with plain make:

```
make -C src                      # serial, no compressed input
make -C src OPENMP=1 ZLIB=1      # GCC, or clang on Linux
make -C src OPENMP=1 ZLIB=1 OMPFLAGS="-Xpreprocessor -fopenmp" OMPLIBS="--lomp"
                                # Apple clang, with libomp installed
```

CMake reports at configure time which of the two it found; the `make` build takes each as a flag. Without OpenMP, `--threads` is accepted but warns that it is being ignored.

2.3 Running

There are two ways to specify a run, and they can be mixed:

```
adze --data mydata.stru --group-col 5 --out-prefix mypops
adze myparamfile.txt --tolerance 0.05
```

The first form takes everything from the command line. The second reads a parameter file in the version 1.0 format — the same keywords, and the same short flags — with command-line options taking precedence over the file. Three sources are consulted, in increasing order of precedence: built-in defaults, then the parameter file if one is given, then the command line.

`--help` lists every option; `--version` prints the version and the method reference. Both work with no data file and no parameter file. `--write-template` writes a commented parameter file that runs as it stands once `DATA_FILE` is filled in.

Only the data file is strictly required. Everything else either has a default or is measured from the data (section 3.3).

3 Input

3.1 Choosing a format

Two input formats are read: the STRUCTURE-like layout that ADZE has always taken (section 3.2), and VCF (section 3.4), optionally compressed with `gzip` or `bgzip`. `--format` selects one; its default, `auto`, reads the file name — anything ending `.vcf`, after an optional `.gz` or `.bgz`, is treated as VCF.

The two paths converge immediately: both reduce each locus to per-grouping allele counts and missing-data tallies, which is all the estimators use, and both lay those counts out identically for the same genotypes. Results therefore do not depend on which format the data arrived in — the same genotypes written both ways produce byte-identical output, which `test/formats.py` checks.

3.2 structure format

This file is expected in the same format used by STRUCTURE: one row per gene copy, so a diploid individual occupies two consecutive rows, preceded by a header row of locus names. Each data row begins with one or more label columns and continues with one allele code per locus. Allele codes are arbitrary tokens — integers, as in the example below, or any other strings that do not contain whitespace.

Figure 1 shows `example/small_data.stru`, distributed with ADZE: five loci, three groupings, and eleven diploid individuals, with missing alleles coded `-9`.

```
D12S1638 D14S1007 D9S1779 D9S1825 D7S2477
1037 87 Pima Mexico AMERICA 126 128 124 -9 142
1037 87 Pima Mexico AMERICA 126 128 142 -9 142
1038 87 Pima Mexico AMERICA 120 124 124 129 142
1038 87 Pima Mexico AMERICA 126 128 152 137 142
...
1357 24 Basque France EUROPE 126 120 124 -9 154
1357 24 Basque France EUROPE 128 122 126 -9 156
...
747 684 Japanese Japan EAST_ASIA 126 -9 124 129 158
747 684 Japanese Japan EAST_ASIA 128 -9 144 135 158
...
```

Figure 1: The distributed example data file, abridged. Row 1 holds the locus names; each subsequent row is one gene copy, with five label columns followed by five allele codes.

3.3 Dimensions are measured, not declared

Version 1.0 required `LOCI`, `DATA_LINES`, `NON_DATA_ROWS` and `NON_DATA_COLS` to be supplied by hand, and validated them against the file. Version 2.0 measures them while reading:

`--loci`
from the number of names in the header row.

`--data-lines`
by counting data rows.

`--non-data-cols`
from the width of the first data row minus the locus count.

`--group-col`
defaults to the last label column.

`--non-data-rows`
defaults to 1, the locus-name row. This is the one value that cannot be inferred, so give it explicitly if the file has additional header rows.

`--max-g`
defaults to the largest standardized sample size the data supports, which is the smallest N_j over all groupings and all *surviving* loci — resolved after missing-data filtering, since dropping loci can raise it.

Any of these may still be given, in which case the declared value is used and a warning is printed if it disagrees with what was measured. `--dry-run` reports the detected layout, the per-grouping sample sizes, the largest feasible `MAX_G` and the number of tuples an analysis would evaluate, then exits without computing anything.

3.4 VCF format

Each record is one locus and each allele index is one allele type, so the reference allele is allele 0 and the alternates follow. A sample contributes as many gene copies as its `GT` field holds, which lets haploid and diploid records sit in the same file, and a `.` is an uncalled copy. A sample's ploidy is the largest number of alleles any of its `GT` fields holds, since that is how many gene copies it could have contributed; a record where it carries fewer — a haploid call in an otherwise diploid file, say — counts the difference as missing, and so tells against that locus under `--tolerance`. Only `GT` is read: no other `FORMAT` field, and no `INFO`, `QUAL` or `FILTER` value, affects the counts, so filtering must be done before ADZE sees the file. Locus names come from the `ID` column, or from `CHROM:POS` where `ID` is `..`.

A VCF names its samples but says nothing about which population each belongs to, so `--samples` is required with VCF input. It takes a file of two whitespace-separated columns, sample name then grouping name:

# sample	grouping
HG00096	GBR
HG00097	GBR
NA18939	JPT

Anything after `#` is a comment and any further columns are ignored, so an existing sample sheet can usually be passed as it stands. Groupings are numbered in the order they first appear in this file, which is the order they appear in the output. A sample in the VCF that the map does not mention is skipped, and a sample the map names that the VCF does not contain is ignored; both are reported as warnings with a count.

Because a sample's gene copies are what the missing-data fraction is measured against, `--tolerance` means the same thing here as for STRUCTURE input: the fraction of a grouping's gene copies that are not called at that locus (section 3.6).

```
adze --data cohort.vcf.gz --samples populations.tsv \
    --tolerance 0.05 --out-prefix cohort
```

NON_DATA_ROWS, NON_DATA_COLS and GROUP_BY_COL describe the STRUCTURE layout and are ignored, with a warning, for VCF input.

3.5 Groupings

With STRUCTURE input, rows are grouped by the value in the `--group-col` label column; with VCF input the grouping comes from `--samples`. In both cases groupings appear in output in the order they are first seen; any label column can be used, so a file annotated with population, region and continent can be analysed at any of those levels without being rewritten, and the same VCF can be analysed at several levels by swapping the map file. `--pops` restricts the analysis to a comma-separated list of groupings and `--exclude-pops` removes them; both change J , and therefore change what “private” means, so they are a way of asking a different question rather than a way of subsetting output.

3.6 Missing data and locus filtering

An allele code equal to `--missing` (default `-9`) is treated as unobserved and reduces N_j at that locus. Because a statistic is reported at a given g only when it is defined at every locus, one locus with a small N_j limits the whole analysis.

`--tolerance  $x$` drops any locus at which some grouping has more than a fraction x of its gene copies missing. Loci dropped this way are listed in the `_deletedloci` file (section 5.4). Filtering is a trade: it discards loci to buy a larger MAX_G, and section 7.1 works through the trade-off on the example data.

The default is $x = 0.1$. Version 1.0's default, $x = 1$, keeps every locus, and on modern data that is rarely what is wanted: a single locus at which one grouping scored nothing drives MAX_G to 1 and leaves that grouping undefined at every g , so a genome-wide run can complete and report nothing. `--tolerance 1` restores the old behaviour for anyone who wants it. A parameter file that declares TOLERANCE is unaffected, and 1.0's files all declare it.

Two conditions make a grouping unusable, and both are now reported by name rather than left to be inferred from empty output. A locus at which no allele is observed in any grouping contributes nothing: such loci are counted and reported, and treated as undefined at every g . A locus at which one particular grouping scored fewer than g gene copies makes that grouping undefined at g — and because the sentinel propagates through the average, one such locus is enough. ADZE therefore reports, per grouping, the fewest copies it scored anywhere, how many loci sit at that floor, and how many it did not score at all, adding an explicit remedy when the floor leaves nothing usable. `--dry-run` reports the same ceiling without computing anything. If filtering removes every locus, ADZE stops with an error rather than reporting statistics over no data.

4 Parameters and options

4.1 Parameter file syntax

A parameter file is a sequence of lines, each a keyword followed by its value:

```
DATA_FILE small_data.stru
GROUP_BY_COL 5
MAX_G 8
TOLERANCE 0.24
# everything after a hash is a comment
```

Anything after # is ignored. The value is the rest of the line with surrounding whitespace removed, so values may contain spaces and may contain other keywords as substrings. Boolean parameters take 0 or 1.

4.2 Reporting a single g

By default every g from 1 to `MAX_G` is reported. `--at-g` asks for one instead:

```
--at-g N
    report only  $g = N$ .
```

```
--at-g max
    report only at whatever MAX_G resolves to, without having to know the number in advance.
```

One flag governs every output: the three statistics, their per-locus companions and their window files all carry the single g , with the same columns as a full run — including the g column — so a consumer reads either the same way.

Two ceilings bind the request: `MAX_G`, which is what the run was asked to sweep, and the smallest number of gene copies scored anywhere, which is the largest g with defined values. A request above the lower of the two is met at that ceiling, with a warning naming the value asked for, the value used and which bound applied. This differs deliberately from an over-large `MAX_G`, which keeps its shape and reports the unreachable rows as undefined: a ladder with `NA` at the top is still a ladder, whereas a single- g report of nothing but `NA` would be no answer at all. A value below 1 is refused rather than rounded up, g being a sample size.

Because the recurrence in g is sequential, the sweep still has to climb to the g being reported — but no further, and the buffers hold no more than that. The saving therefore scales with how far below `MAX_G` the request sits: on 20 000 loci in 12 groupings with all 66 pairwise tuples and `MAX_G` at 33, `--at-g 2` ran in 0.58 s against 6.72 s for the full ladder, while `--at-g 33` saved 3%, the climb being the whole ladder anyway.

4.3 Option reference

The following is generated from the option table in the source, so it lists exactly what the program accepts. Each entry gives the long option, the version 1.0 short flag where one exists, and the parameter-file keyword where one exists.

4.3.1 Input options

- data** *FILE*, **-f**, DATA_FILE (**paramfile**)
string
genotype file: STRUCTURE layout or VCF, optionally gzipped (required)
- format** *FMT*, FORMAT (**paramfile**)
string
input format: auto, structure or vcf (default auto, by file extension)
- samples** *FILE*, SAMPLE_FILE (**paramfile**)
string
sample-to-grouping map, required for VCF input (two columns: sample grouping)
- loci-map** *FILE*, LOCI_MAP (**paramfile**)
string
locus coordinates for STRUCTURE input (three columns: locus chromosome position)
- max-g** *N*, **-g**, MAX_G (**paramfile**)
int
largest standardized sample size (default: the largest the data supports)
- non-data-rows** *N*, **-nr**, NON_DATA_ROWS (**paramfile**)
int
header rows before the genotypes (default 1)
- non-data-cols** *N*, **-nc**, NON_DATA_COLS (**paramfile**)
int
label columns before the genotypes (default: detected)
- loci** *N*, **-l**, LOCI (**paramfile**)
int
number of loci (default: detected from the header row)
- data-lines** *N*, **-d**, DATA_LINES (**paramfile**)
int
number of data rows (default: detected)
- group-col** *N*, **-s**, GROUP_BY_COL (**paramfile**)
int
which label column names the grouping (default: the last one)
- missing** *STR*, **-m**, MISSING (**paramfile**)
string
code for a missing allele (default -9)
- pops** *LIST*
string
analyse only these groupings (comma-separated)
- exclude-pops** *LIST*
string
analyse everything except these groupings

4.3.2 Analysis options

--window-bp *N*, WINDOW_BP (**paramfile**)
int
sliding windows *N* basepairs wide (needs locus coordinates)

--window-loci *N*, WINDOW_LOCI (**paramfile**)
int
sliding windows *N* loci wide (surviving loci; needs coordinates for chromosomes)

--step-bp *N*, STEP_BP (**paramfile**)
int
advance basepair windows by *N* (default: the window width, i.e. no overlap)

--step-loci *N*, STEP_LOCI (**paramfile**)
int
advance locus windows by *N* loci (default: the window width)

--min-window-loci *N*, MIN_WINDOW_LOCI (**paramfile**)
int
do not report a window holding fewer than *N* loci (default 1)

--at-g *N/max*, AT_G (**paramfile**)
string
report only this *g*, or max for whatever MAX_G resolves to (default: every *g*)

--stat *LIST*
string
which statistics to compute: richness,private,tuples (default: richness,private)

--tolerance *X*, -t, TOLERANCE (**paramfile**)
float
drop a locus if any grouping exceeds this missing fraction (default 0.1; 1 keeps all)

--combinations, -c, COMB (**paramfile**)
boolean
also compute private alleles of grouping tuples

--tuples-k *LIST*, -k, K_RANGE (**paramfile**)
string
tuple sizes for -combinations, e.g. 2 or 1-3 or 1,3,5-7

--tuples *FILE*
string
compute only the named tuples in *FILE* (one per line) instead of every *k*-subset

--threads *N*
int
worker threads (default 1; requires an OpenMP build)

4.3.3 Output options

--out-prefix *PREFIX*
string
compose output names from PREFIX (default adze)

--out-richness *FILE*, **-r**, *R_OUT* (**paramfile**)
string
allelic richness output file

--out-private *FILE*, **-p**, *P_OUT* (**paramfile**)
string
private allelic richness output file

--out-tuples *FILE*, **-o**, *C_OUT* (**paramfile**)
string
tuple output file (one per tuple size, suffixed *_k*)

--full-richness, **-fr**, *FULL_R* (**paramfile**)
boolean
also write per-locus allelic richness

--full-private, **-fp**, *FULL_P* (**paramfile**)
boolean
also write per-locus private allelic richness

--full-tuples, **-fc**, *FULL_C* (**paramfile**)
boolean
also write per-locus tuple values

--tsv
boolean
tab-separated output with a header row and NA for undefined values

4.3.4 Other options

--params *FILE*
string
read parameters from FILE (also accepted as the first argument)

--dry-run
boolean
report the detected layout, groupings and feasible MAX_G, then stop

--progress, **-pp**, *PRINT_PROGRESS* (**paramfile**)
boolean
progress bars (default: on when stderr is a terminal)

--quiet
boolean
suppress progress and informational messages

`--tolerance-only, -tnocalc, TNC (paramfile)`
boolean
 apply the missing-data filter, report it, and stop

`--no-check, -skipchk, SKIP_CHK (paramfile)`
boolean
 accepted for compatibility; validation is now free and always on

5 Output

By default ADZE writes `adze.richness`, `adze.private` and `adze.summary.txt` in the current directory; `--out-prefix` replaces the `adze` stem. Giving an explicit name with `--out-richness`, `--out-private` or `--out-tuples` uses that name literally and appends version 1.0's suffixes (`_summary`, `_fulldata`, `_deletedloci`) to it.

Results go to files; progress, warnings and errors go to standard error, so redirecting standard output captures results alone. `--quiet` silences the diagnostics. Progress bars are drawn only when standard error is a terminal.

5.1 Allelic richness and private allelic richness

One row per grouping per g : the grouping name, the g at which the statistics were computed, the number of loci used, and then the mean, variance and standard error across those loci (Equations 6–8). Groupings are separated by a blank line.

```
AMERICA 2 5 1.47429 0.0886378 0.133145
AMERICA 3 5 1.76929 0.264997 0.230216

EUROPE 2 5 1.68 0.0208889 0.0646357
```

5.2 Private allelic richness of tuples

The same layout, with the tuple's grouping names in place of the single name. With `--tuples-k` the tuple size k is appended to the file name, one file per k ; with `--tuples` the named tuples all go to one file.

5.3 Locus-specific output

`--full-richness`, `--full-private` and `--full-tuples` add a `_fulldata` file that carries, in addition to the summary columns, the statistic at every locus used. The first line names the columns, including one column per locus.

5.4 Filtered loci

When `--tolerance` is below 1, a `_deletedloci` file records how many loci were dropped, the threshold applied, and the name of each dropped locus.

5.5 Run summary

A `_summary` (or `.summary.txt`) file is written on every run. It echoes the parameters actually used — including values that were measured rather than declared, and `MAX_G` when it was resolved from the data — and records the elapsed time at which each phase finished.

5.6 Tab-separated output

`--tsv` writes the result files with a header row, tab separators, and `NA` in the value columns where a statistic is undefined at that g . Without it, the legacy format is used: no header, space separators, and rows that are undefined at a given g are omitted entirely. The legacy format remains the default so that existing scripts keep working, but `--tsv` is preferable for new work, because a consumer can distinguish “not computable here” from “not run”.

5.7 Exit status

0	success
2	usage error: unknown option, missing value, contradictory request
3	I/O error: a file could not be opened, read or written
4	data error: the file disagrees with itself or with the parameters

6 Sliding windows

Every statistic in section 1 can be reported window by window along the genome instead of, or as well as, once for the whole dataset.

6.1 What a windowed statistic is here

For each window, ADZE takes the loci inside it and reports the same mean, variance and standard error over those loci that it would report over all of them. Nothing about the estimators changes: $\hat{\alpha}_{jg}$ and $\hat{\pi}_{jg}$ are expectations for a single locus, and a window is simply a different set to average over.

Two consequences are worth stating plainly. The statistic has no haplotype or linkage component — it does not know that the loci in a window are near each other, only that they were grouped together — so a window here is a smoothing device, not a measure of local haplotype structure. And the variance and standard error are taken across the loci in the window, so with few loci they are noisy; with one locus the variance is undefined and ADZE writes `nan`. Use `--min-window-loci` to set a floor.

6.2 Coordinates

A window is a range of the genome, so every locus has to be placed on it. With VCF input the coordinates come from `CHROM` and `POS`. The `STRUCTURE` layout carries none, so `--loci-map` supplies them:

# locus	chromosome	position
L0	chr1	5000
L1	chr1	10000
L2	chr2	5000

Whitespace-separated, `#` starts a comment, further columns are ignored. Every locus in the data must appear: a window over loci whose positions are unknown would be a window in name only, so a missing coordinate stops the run rather than producing a quietly meaningless scan. Loci must arrive in genome order — positions increasing within a chromosome, and each chromosome in one block — and ADZE says which locus breaks the order rather than sorting a dataset it is streaming.

6.3 Defining the windows

A window is measured either in basepairs or in loci, never both:

--window-bp *N*

windows *N* basepairs wide, anchored at position 1 and advancing by **--step-bp**. The intervals do not depend on where the loci fall, so the same region gives the same windows in every run and two datasets can be compared window by window. How many loci a window holds varies with locus density.

--window-loci *N*

windows of *N* surviving loci, advancing by **--step-loci**. Each window carries the same weight in the averaging; its width in basepairs varies with density. The interval reported is the span of the loci it holds.

Each step defaults to its window width, so the default is a non-overlapping tiling; a step smaller than the width is the sliding case. No window crosses a chromosome. Windows are laid out over the loci that survive the missing-data filter (section 3.6), so a locus dropped there is absent from every window, and a locus-counted window of *N* means *N* surviving loci.

--min-window-loci *N* suppresses windows holding fewer than *N* loci; the default of 1 reports every window that holds anything. Windows holding no loci at all are never reported.

6.4 *g* is fixed across the scan

MAX_G is resolved genome-wide, as it is for the whole-dataset statistics: the largest standardized sample size every grouping supports at every surviving locus. It is not re-resolved per window. A window in a sample-rich region is therefore not reported at a larger *g* than the rest of the scan, which is what makes windows comparable with each other — the point of a scan. Since a set containing a locus that cannot reach *g* is undefined at that *g*, this also means that under the default every window is defined at every reported *g*. If you raise **MAX_G** past what some locus supports, the windows containing such loci are written as **NA** rather than dropped, so the file keeps one row per window per *g*.

6.5 Output

Each requested statistic writes an extra file named by appending **_windows**: with **--out-prefix scan**, **scan.richness** gains **scan.richness_windows**. The whole-dataset files are written exactly as before; asking for windows never changes them.

Window files always carry a header and are always tab-separated, whatever **--tsv** says — they are new output with no version 1.0 consumer to stay compatible with, and a scan is read by machine. The columns are

CHROM	START	END	POP_GROUPING	G	NUM_LOCI	MEAN	VAR	STD_ERR
-------	-------	-----	--------------	---	----------	------	-----	---------

with **TUPLE** in place of **POP_GROUPING** in the tuple file, where the grouping names are joined with commas so the label stays one field. Rows are ordered by grouping, then by *g*, then along the genome, so the track for one grouping at one *g* is a contiguous block.

A scan is usually read at one *g*, and **--at-g** (section 4.2) is the economical way to ask for it: the file holds one row per window per grouping instead of one per window per grouping per *g*. On the scan below that is the difference between 912 rows and 48.

Windowed tuple output is large: one row per window per g per tuple. `--dry-run` reports the row count each statistic would write, along with the window count and how many windows fell below the threshold, before anything is computed.

7 Examples

7.1 The distributed example data set

The data set in Figure 1 has five loci and three groupings. Running it with nothing but the data file and the grouping column,

```
adze --data small_data.stru --group-col 5 --out-prefix small
```

reports on standard error

```
Read 22 gene copies at 5 loci in 3 groupings (d:h:m:s 0:0:0:0.00)
2 loci have at least one grouping with more than 10% missing data.
MAX_G not given; using 6, the largest the 3 surviving loci support.
```

Two of the five loci exceed the default tolerance of 0.1 and are dropped, and the analysis runs to $g = 6$ over the three that remain. Asking instead for every locus,

```
adze --data small_data.stru --group-col 5 --out-prefix keep --tolerance 1
```

```
MAX_G not given; using 3, the largest the 5 surviving loci support.
```

halves the standardized sample size. Three is low for 22 gene copies, and the reason is missing data rather than sample size: the EUROPE grouping has six gene copies in total, but at locus D9S1825 three of those six are missing, leaving $N_j = 3$. D9S1825 is also missing 25% of the AMERICA grouping, and D14S1007 is missing 25% of EAST_ASIA — which is why the default drops exactly those two.

Which run to prefer is a judgement about the data: the second uses all five loci but compares populations only at $g \leq 3$; the first compares them over a much wider range of g using three loci. The default leans towards the larger g , on the grounds that a locus one grouping barely scored constrains every grouping.

The parameter file distributed with the example, `small_paramfile.txt`, runs the same analysis in version 1.0's style, declaring all four dimensions along with `MAX_G 8` and `TOLERANCE 0.2`:

```
adze small_paramfile.txt
```

Its results agree with the run above wherever the two overlap, and the private allelic richness files are identical. The richness files are not: declaring `MAX_G 8` reports AMERICA and EAST_ASIA at $g = 7$ and 8, which their own sample sizes reach, whereas the default stops at the largest g that *every* grouping supports. Private allelic richness stops at 6 in both, because no allele can be private at a g that some grouping cannot reach.

7.2 A larger data set

The next example uses the data of Rosenberg et al. (2002): 1056 individuals from 52 populations typed at 377 microsatellite loci, available at <https://rosenberglab.stanford.edu/diversity.html>. Because computing private alleles for all combinations at the population level would be prohibitive, the published analysis groups the individuals by region instead, which is column 5 of that file's labels and reduces the number of groupings from 52 to 7.

The version 1.0 parameter file for this analysis (Figure 3 of the version 1.0 manual) declared MAX_G 70, DATA_LINES 2112, LOCI 377, NON_DATA_ROWS 1, NON_DATA_COLS 5, GROUP_BY_COL 5, K_RANGE 2-6 and TOLERANCE 1. In version 2.0 the four dimensions are measured from the file, so the same run is

```
adze --data diversitydata.stru --group-col 5 --max-g 70 \
    --stat richness,private,tuples --tuples-k 2-6 \
    --out-prefix hgdp --threads 8
```

Figures 2–4 show the published analysis of these data: mean allelic richness against g , mean private allelic richness against g , and mean private allelic richness for combinations of six of the seven regions — that is, “missing” allelic richness, Equation 5. These figures are reproduced from the version 1.0 manual and were produced with version 1.0; version 2.0 computes the same values.

Africa carries both the highest allelic richness and, by a wide margin, the highest private allelic richness, while the Americas and Oceania have the highest missing allelic richness — the alleles absent there but present everywhere else.

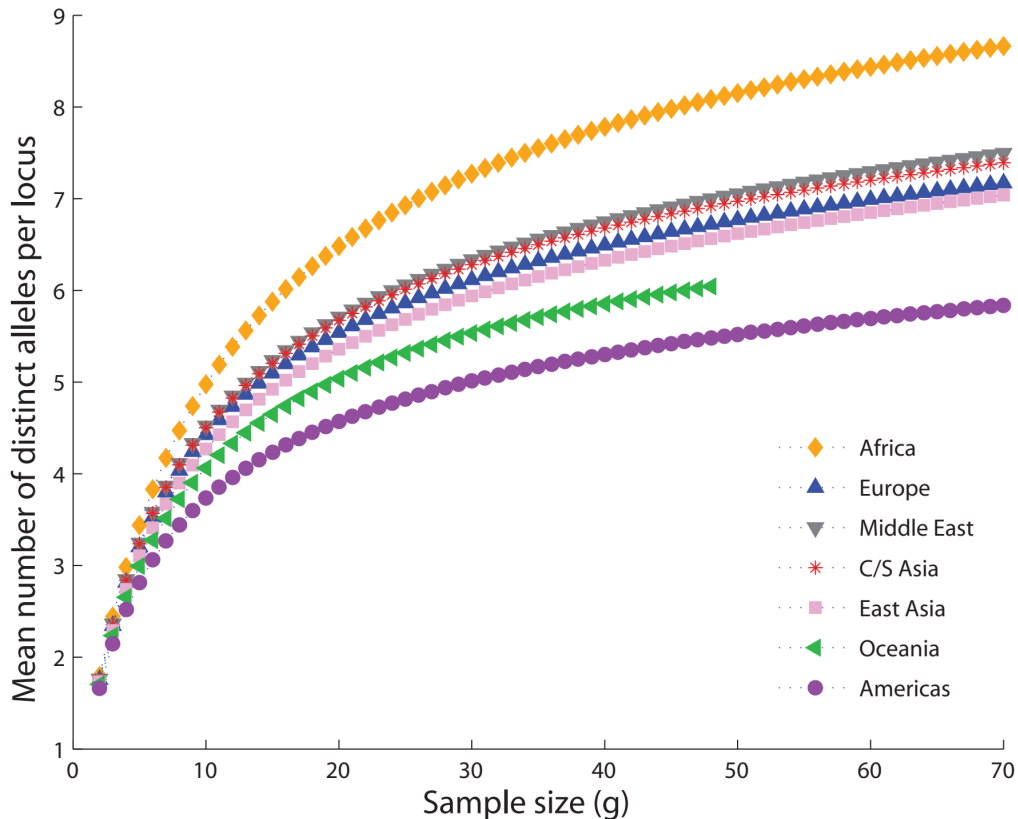


Figure 2: The mean number of distinct alleles per locus as a function of standardized sample size for seven major geographic regions.

7.3 Naming the tuples of interest

A full sweep over tuple sizes evaluates $\binom{J}{k}$ combinations for each k , which is 2^J tuples if every k is requested. When only a few groupings matter, list them in a file, one tuple per line:

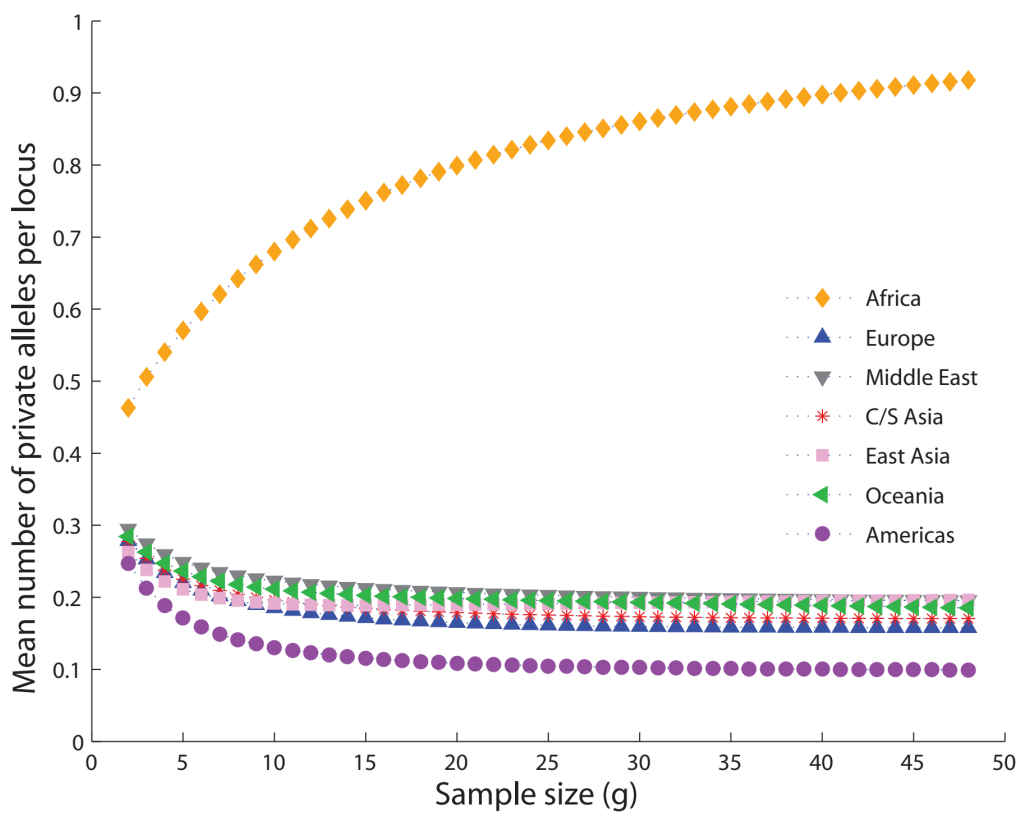


Figure 3: The mean number of private alleles per locus as a function of standardized sample size for seven major geographic regions.

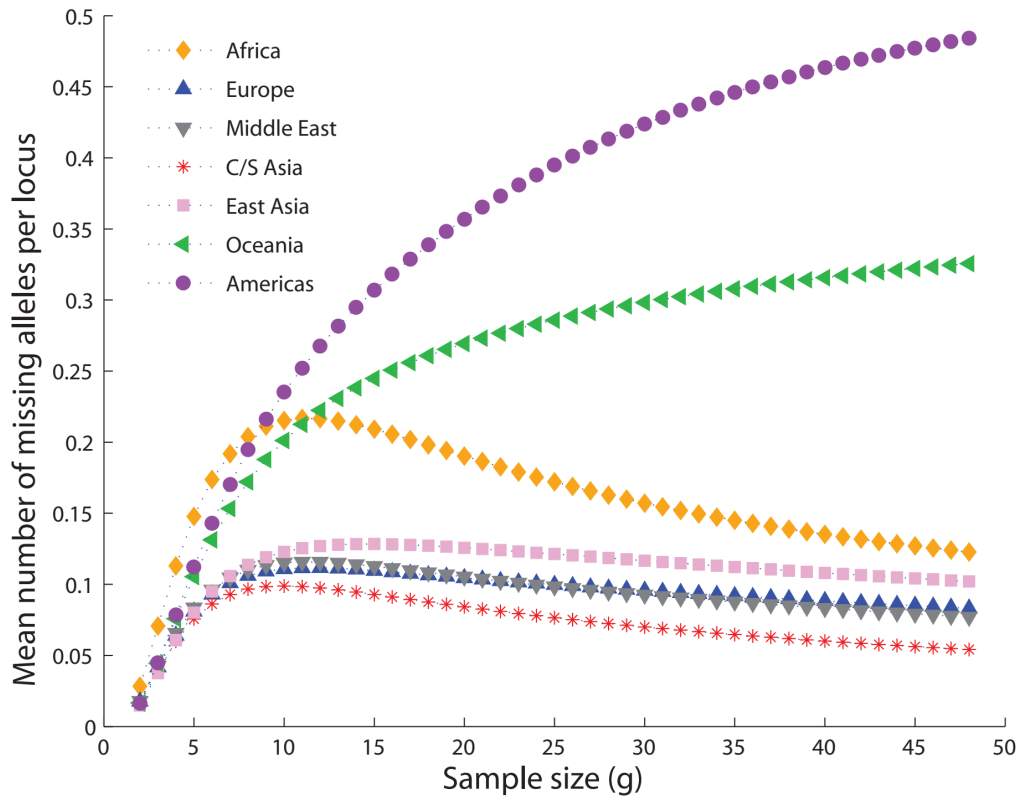


Figure 4: The mean number of missing alleles per locus for seven major geographic regions (private to combinations of six of the seven regions) as a function of standardized sample size.

```
# tuples.txt
AMERICA, EUROPE
EAST_ASIA
AMERICA, EUROPE, EAST_ASIA

adze --data small_data.stru --group-col 5 --tuples tuples.txt \
    --stat tuples --out-prefix regions
```

Each line is a comma- or space-separated list of grouping names as they appear in the data file; # starts a comment. This computes only the tuples named, at any mixture of sizes.

7.4 A genome scan

The generator that drives the test suite writes coordinates as well as genotypes, so a scan can be reproduced from the repository alone. This makes 60 loci spaced 5 kb apart over two chromosomes, in four groupings of twelve diploid individuals:

```
python3 -c "import sys; sys.path.insert(0,'test'); import gen_data
gen_data.write_pair('scan', npops=4, nind=12, nloci=60, nall=6,
    missing=0.03, seed=3, chroms=2, spacing=5000)"
```

Ask what a 50 kb window stepping 25 kb would cover before running it:

```
adze --data scan.vcf --samples scan.samples \
    --window-bp 50000 --step-bp 25000 --dry-run
```

```
Largest feasible MAX_G: 20
MAX_G to be used:      the largest feasible
(TOLERANCE 0.1 will drop loci first, which can raise both numbers)
Windows:              12 of 50000 bp, step 25000
  loci per window: 4-10
  richness rows:    960
  private rows:    960
```

Twelve windows and, before filtering, g running to 20, every grouping having 24 gene copies and the sparsest locus carrying 20. The projection is deliberately conservative about its own arithmetic: filtering happens first and raises both the reachable g and the row count, as the run itself then reports. Then run it, with a floor on how thin a window may be:

```
adze --data scan.vcf --samples scan.samples \
    --window-bp 50000 --step-bp 25000 --min-window-loci 3 \
    --out-prefix scan
```

```
7 loci have at least one grouping with more than 10% missing data.
Laid out 12 windows over 60 loci.
MAX_G not given; using 22, the largest the 53 surviving loci support.
```

The filter drops seven of the sixty loci, which raises the reachable g from 20 to 22 and the output to 1056 rows — more than a scan is usually read at. Adding `--at-g 10` asks for one g and gives 48, one row per window per grouping:

CHROM	START	END	POP_GROUPING	G	NUM_LOCI	MEAN	VAR	STD_ERR
chr1	1	50000	POPO	10	8	3.45471	0.0842792	0.10264
chr1	25001	75000	POPO	10	8	3.56664	0.0605046	0.0869659

The consecutive windows overlap by 25 kb, so each interior locus contributes to two of them. `scan.richness` holds the whole-dataset figures unchanged alongside, and the rows here are the same bytes the full ladder gives at $g = 10$.

8 Performance and threading

The per-locus loops of all three statistics can be spread over threads with `--threads` in an OpenMP build. Loci are processed independently and reduced in locus order, so the output does not depend on the thread count: the result files are bit-identical at any `--threads` setting.

Compressed input is decompressed as it streams, so a `.vcf.gz` costs more processor time to read than the same data uncompressed, and usually less wall-clock time on a busy filesystem, since there is far less to transfer.

Table 1 gives the effect of the version 2.0 work on the phases that dominate large runs, measured against a version 1.0 build on the same inputs. The costs that grew fastest in version 1.0 were locus filtering (quadratic in the number of loci) and the sweep over g (quadratic in `MAX_G`); both are now linear.

Phase	Condition	Version 1.0	Version 2.0
Locus filtering	8000 loci, 4623 dropped	5.97 s	<0.01 s
Richness + private	1000 loci, <code>MAX_G</code> 80	4.87 s	0.24 s
Tuples	12 groupings, 495 tuples	4.33 s	0.67 s
Reading	64 MB input	3.62 s	0.70 s
Peak memory	64 MB input	404 MiB	21 MiB
Statistics, 8 threads	2000 loci, <code>MAX_G</code> 150	—	4.0×

Table 1: Measured on an 8-core machine, best of two runs. Version 1.0 held every genotype in memory, so its footprint grew with sample size; version 2.0 keeps allele counts only, and its footprint is flat in the number of gene copies.

9 Changes from version 1.0

9.1 Compatibility

Parameter files written for version 1.0 work unchanged, as do the short flags. One default differs: `--tolerance` is 0.1 rather than 1, so a run that names no tolerance now drops loci that version 1.0 kept. Every 1.0 parameter file declares `TOLERANCE` and is therefore unaffected; `--tolerance 1` restores the old behaviour explicitly (section 3.6). Explicitly named output files keep version 1.0’s literal suffixes, and the default result format is unchanged. `SKIP_CHK` and `--no-check` are still accepted but do nothing, because validation now costs nothing: it happens during the single read pass.

9.2 Interface

- Options can be given without a parameter file, and `--help` and `--version` work with no arguments at all. A bare invocation prints usage; version 1.0 wrote a template parameter file into the working directory.
- The parameter-file parser reads a keyword and the rest of the line. Version 1.0 searched each line for any keyword as a substring, so a value that contained a keyword — `R.OUT out_LOCI_richness` — was rejected, and values could not contain spaces.
- Dimensions are measured rather than declared (section 3.3).
- Diagnostics and progress go to standard error; exit status distinguishes usage, I/O and data errors.
- VCF input, plain or `gzip/bgzip` compressed, via `--format` and `--samples` (section 3.4). Version 1.0 read only the uncompressed STRUCTURE layout.
- The `--tolerance` default is 0.1 rather than 1 (section 3.6), and the groupings a retained locus leaves unusable are named with their locus counts rather than showing up as empty result files.
- The `--tolerance` default is 0.1 rather than 1 (section 3.6), and the groupings that a retained locus leaves unusable are named, with their locus counts, instead of showing up as empty result files.
- Every statistic is now also reported at $g = 1$ (section 1.5); version 1.0's sweeps started at $g = 2$. The default output therefore carries one extra row per grouping, which is the one respect in which it is not what 1.0 wrote.
- `--at-g` reports a single g instead of the whole ladder (section 4.2), clamping an over-large request to the largest reachable g with a warning.
- Sliding-window statistics (section 6): `--window-bp` or `--window-loci` with a matching step, `--min-window-loci`, and `--loci-map` to give STRUCTURE input the coordinates a window needs. Version 1.0 reported each statistic once for the whole dataset.
- New: `--out-prefix`, `--stat`, `--pops`, `--exclude-pops`, `--tuples`, `--tsv`, `--dry-run`, `--threads`, `--quiet`.

9.3 Behaviour

Estimates are unchanged. The following differ, in each case because version 1.0 gave an answer that could not be used:

1. A locus with no observed allele in any grouping is reported as undefined, with a warning. Version 1.0 stopped with a segmentation fault.
2. Filtering that removes every locus is an error. Version 1.0 continued and printed `nan` rows over zero loci.
3. A data row whose column count is wrong is always an error. With checking disabled, version 1.0 would misread it silently.

4. The variance of a single locus is reported as `nan` rather than the result of dividing by zero.
5. Reported times are elapsed wall-clock times with two decimals. Version 1.0 reported processor time truncated to whole seconds, which is why the example run in the version 1.0 manual reports 0:0:0:0 throughout.

9.4 Verification

Every change was checked with a differential test that runs version 2.0 and a version 1.0 build over the same inputs and compares every output file byte for byte: nine synthetic data sets — spanning allele-count range, missingness, a single locus, a single grouping, unequal sample sizes, haploid data, and a locus with no observed alleles — crossed with nine parameter settings, plus the distributed example. All 69 comparable cases are byte-identical; the remaining cases are those in which version 1.0 crashes or reports statistics over zero loci, and so has no output to compare. The test is in `test/regress.py` and needs a version 1.0 binary to compare against.

Since $g = 1$ was added, that comparison is stated more precisely: 1.0 cannot produce a $g = 1$ row, so whole files no longer match. Every row at $g \geq 2$ is compared byte-for-byte, which is the guarantee that matters, and the $g = 1$ rows are accounted for separately — each results file must carry exactly one per grouping or tuple, so a build that quietly stopped writing them fails.

The windowed statistics have no version 1.0 counterpart either, so `test/windows.py` checks them against the layout they claim to cover: one window over a single-chromosome dataset must reproduce the whole-dataset output exactly, every window’s mean and variance must match a recomputation from the per-locus columns, the windows must tile as asked and stay inside a chromosome, basepair and locus windows must agree on evenly spaced loci, and the inputs a window cannot be defined over must be refused.

VCF input has no version 1.0 counterpart, so it is checked differently: `test/formats.py` writes one set of genotypes as STRUCTURE, as VCF, and as compressed VCF, and requires the three runs to agree byte-for-byte. Because both readers order allele slots by the same rule, the comparison is exact rather than approximate — any difference at all is a real one, not a change in the order of a floating-point sum.

References

- Hurlbert, S. H. 1971. The nonconcept of species diversity: a critique and alternative parameters. *Ecology* 52: 577–586.
- Kalinowski, S. T. 2004. Counting alleles with rarefaction: private alleles and hierarchical sampling designs. *Conservation Genetics* 5: 539–543.
- Kalinowski, S. T. 2005. HP-RARE 1.0: a computer program for performing rarefaction on measures of allelic richness. *Molecular Ecology Notes* 5: 187–189.
- Petit, R. J., A. El Mousadik, and O. Pons. 1998. Identifying populations for conservation on the basis of genetic markers. *Conservation Biology* 12: 844–855.
- Rosenberg, N. A., J. K. Pritchard, J. L. Weber, H. M. Cann, K. K. Kidd, L. A. Zhivotovsky, and M. W. Feldman. 2002. Genetic structure of human populations. *Science* 298: 2381–2385.
- Szpiech, Z. A., M. Jakobsson, and N. A. Rosenberg. 2008. ADZE: a rarefaction approach for counting alleles private to combinations of populations. *Bioinformatics* 24: 2498–2504.